

Apache Spark: The Spark-Suite

A modular
playbook for
modern data
workloads

Εργ. Βοηθός: Αντώνης Τζιόρβας
Email: atzio@unipi.gr

Spark SQL Launchpad - Load ~1.5M rows in a second

- ❖ Bring a large, real-world dataset into a Spark DataFrame.
- ❖ CSV is a common interchange format; Spark's CSV reader handles schema inference, header detection and null handling automatically.
- ❖ Use `spark.read.option(...).csv(...)` with appropriate options and verify the schema before any transformation.

```
file_path = "/to/data"
trips = spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .option("nullValue", "")
    .csv(file_path)

print(df.printSchema())
```

DataFrames vs. RDDs – Choosing the Right Abstraction

- ❖ DataFrames provide a relational view with Catalyst optimizations; RDDs expose low-level transformations.
- ❖ DataFrames give you automatic query optimization, columnar storage and easier API for analysts; RDDs are needed for custom, non-relational logic or when fine-grained control of partitioning is required.
- ❖ Start with DataFrames for most ETL and ML tasks; fall back to RDDs only when you need custom partitions, complex map-side joins or when integrating non-SQL libraries.

DSL Query – Total Fare by Borough

- ❖ Compute the sum of taxi fares per borough to understand revenue distribution.
- ❖ Aggregations are the backbone of analytical dashboards; grouping reduces millions of rows to a handful of summary records.
- ❖ Use `groupBy` + `agg` with Spark's built-in `sum` function; Catalyst will push the aggregation down to the data source when possible.

```
fare_by_borough = trips
    .groupBy("borough")
    .agg(
        F.sum("fare")
        .alias("total_fare")
    )

fare_by_borough.show()
```

SQL String Query – Translating Business Question to SQL

- ❖ Express the same revenue-by-borough question using pure SQL syntax.
- ❖ **Why:** Many analysts are comfortable with SQL; Spark SQL lets you run the same logical plan without rewriting code.
- ❖ Register the DataFrame as a temporary view and run a SQL statement; Spark parses, optimises and executes it exactly as the DSL version.

```
df.createOrReplaceTempView('taxi')
```

```
spark
```

```
.sql('SELECT borough, SUM(fare) AS total_fare FROM taxi GROUP BY  
borough')  
.show()
```

Structured Streaming Demo – Real-time Word Count

- ❖ Process an unbounded stream of text lines coming from a socket.
- ❖ Streaming enables low-latency insights (e.g., monitoring logs, IoT telemetry) while reusing the same DataFrame API as batch.
- ❖ Define a streaming DataFrame, apply the same `flatMap` → `groupBy` → `count` pipeline, and start a query with a console sink; Spark manages micro-batch execution automatically.

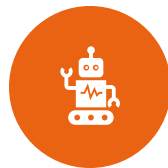
```
lines = spark.readStream
    .format("socket")
    .option("host", "localhost")
    .option("port", 9999)
    .load()

words = lines.select(
    F.explode(F.split(F.col("value"), " "))
    .alias("word")
)

counts = words.groupBy("word").count()

query = counts.writeStream
    .outputMode("complete")
    .format("console")
    .start()
```

MLlib Pipeline Overview – End-to-end Workflow



A pipeline chains data-preparation stages with a machine-learning estimator.



Pipelines guarantee reproducibility, simplify hyper-parameter tuning and let spark treat the whole workflow as a single model object.



Assemble stages (indexers, vectorizers, estimator) into a pipeline; call fit on training data and transform on new data.

Feature Engineering – Encoding Categorical Variables

- ❖ Convert string columns (e.g., gender, city) into numeric vectors.
- ❖ Most ML algorithms operate on numeric input; proper encoding preserves ordinal information (if any) and avoids the “dummy variable trap”.
- ❖ Apply `StringIndexer` to create integer indices, then `OneHotEncoder` to produce a sparse vector; both stages can be inserted into a pipeline for seamless reuse.

```
indexer = StringIndexer(  
    inputCol="city",  
    outputCol="city_idx"  
)  
  
encoder = OneHotEncoder(  
    inputCol="city_idx",  
    outputCol="city_vec"  
)  
  
pipeline = Pipeline(stages=[indexer, encoder])  
  
model = pipeline.fit(df)  
  
model.transform(df).show(5)
```

Logistic Regression – Interpreting Coefficients

- ❖ Binary classification model that predicts the log-odds of the positive class.
- ❖ Coefficients directly indicate the impact of each feature on the odds; useful for explainability and regulatory compliance.
- ❖ After training, retrieve `model.coefficients` and `model.intercept`; convert to odds ratios with `exp(coef)` to communicate effect size to stakeholders.

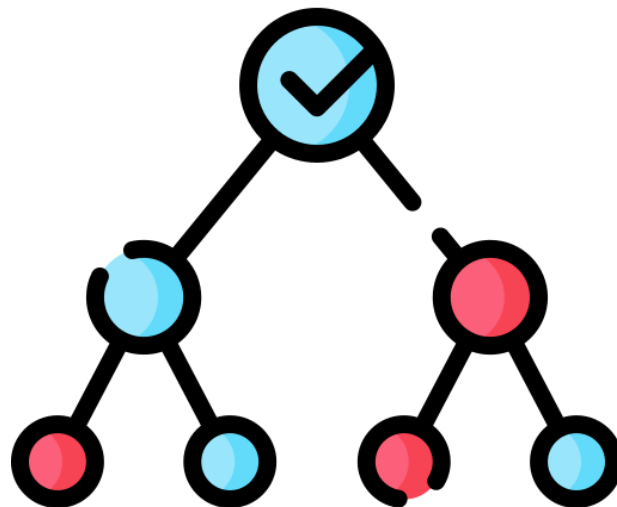
```
lr = LogisticRegression(  
    labelCol="label",  
    featuresCol="features"  
)  
  
model = lr.fit(train)  
  
odds = [exp(c) for c in 3.coefficients]  
  
print("Odds ratios:", odds)
```

Support Vector Machine – Kernel Trick & Use-Cases

- ❖ SVM finds a hyper-plane that maximizes the margin between classes.
- ❖ With a kernel (RBF, polynomial) you can separate data that is not linearly separable without explicitly mapping to high-dimensional space.
- ❖ Choose `kernel='rbf'` for non-linear problems; tune `c` (regularization) and `gamma` (kernel width) via cross-validation to balance bias-variance.

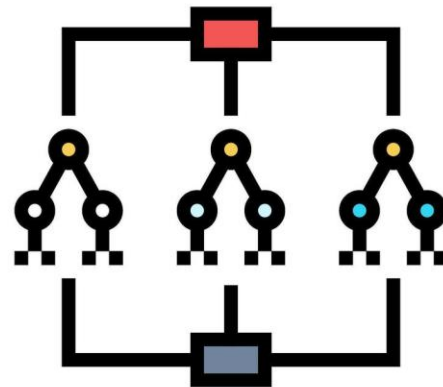
Decision Trees – Depth, Interpretability & Overfitting

- ❖ Tree-based model that recursively splits on feature thresholds.
- ❖ Provides a clear decision path that can be visualized and audited; however, deep trees tend to over-fit noisy data.
- ❖ Control over-fitting by setting `maxDepth`, `minInstancesPerNode` and `minInfoGain`; prune after training if necessary.



Random Forest – Ensemble Benefits & Hyper-parameters

- ❖ An ensemble of decorrelated decision trees built on bootstrap samples.
- ❖ Reduces variance dramatically while preserving interpretability through feature-importance scores.
- ❖ Tune `n_estimators`, `maxDepth`, and `maxFeatures`; use out-of-bag error (`oobScore=True`) for a quick performance estimate without a validation set.



Hyper-parameter Tuning – Grid, Random & Bayesian Strategies

- ❖ Systematic search of model parameters to maximize a chosen metric.
- ❖ Exhaustive grid search guarantees coverage but is costly; random search often finds good regions faster; Bayesian optimization uses a surrogate model to focus on promising areas.
- ❖ Implement `ParamGridBuilder` for grid, `RandomizedSearchCV` for random, and `Optuna` for Bayesian; respect compute budget by limiting iterations or time.

```
param_grid = ParamGridBuilder()
    .addGrid(lr.regParam, [0.01, 0.1, 1.0])

    .addGrid(lr.elasticNetParam, [0.0, 0.5, 1.0])

    .build()

cv = CrossValidator(
    estimator=lr,

    estimatorParamMaps=param_grid,

    evaluator=BinaryClassificationEvaluator(),

    numFolds=5,

    )
```

Model Evaluation – Selecting Metrics for Imbalanced Data

- ❖ Quantify predictive performance beyond simple accuracy.
- ❖ In skewed datasets, accuracy can be misleading; metrics like precision, recall, F1, ROC-AUC, and PR-AUC highlight different aspects of error.
- ❖ Choose metric aligned with business cost (e.g., false negatives expensive → maximise recall). Use `BinaryClassificationEvaluator(metricName='areaUnderPR')` for PR-AUC.

Performance Comparison – Visualising Trade-offs

- ❖ Display a side-by-side view of training time, inference latency and evaluation scores for each model.
- ❖ Stakeholders need to understand resource implications; a slower model may be acceptable if it yields higher recall.
- ❖ Collect timing with `time.time()` around fit and transform; plot bar charts for time and line charts for metrics; annotate with decision recommendations.

GraphFrames— When Graph Analysis Adds Value



GraphFrames provides a dataframe-based API for vertices and edges, enabling graph algorithms within spark.



Certain problems (social network influence, recommendation via link analysis, fraud detection) are naturally expressed as traversals rather than joins.



Define vertices and edges dataframes, instantiate graphframe, then call algorithms like *pagerank* or *connectedcomponents*.

PageRank – Understanding Importance Scores

- ❖ Iterative algorithm that assigns each vertex a score proportional to the scores of inbound neighbors.
- ❖ Identifies influential nodes (e.g., hub airports, popular webpages) without manual heuristics.
- ❖ Run `graph.pageRank(...)`; inspect the `pagerank` column and rank vertices to reveal top influencers.

```
from graphframes import GraphFrame
```

```
vertices = spark.createDataFrame(  
    [("ATL",), ("ORD",), ("LAX",)],  
    ["id"]  
)  
edges = spark.createDataFrame(  
    [("ATL", "ORD"), ("ORD", "LAX"), ("LAX", "ATL")],  
    ["src", "dst"],  
)  
g = GraphFrame(vertices, edges)  
g.vertices.show()  
g.edges.show()
```

Connected Components – Detecting Isolated Sub-graphs

- ❖ Labels each vertex with a component identifier representing its connected sub-graph.
- ❖ Useful for spotting data quality issues (orphan records) or for segmenting networks into independent clusters.
- ❖ Execute `graph.connectedComponents()` and aggregate by component to count vertices per component.

Integrating Graphs with ML – Cluster-Based Feature Enrichment

- ❖ Use graph-derived attributes (e.g., *PageRank*, component id) as additional features for downstream classifiers.
- ❖ Enriches the feature space with relational information that pure tabular data cannot capture, often boosting predictive power.
- ❖ Join the vertex DataFrame (with graph metrics) back to the original transaction DataFrame on a key, then feed the extended DataFrame into an ML pipeline.

```
vertex_features = graph.vertices
    .select('id','pagerank','component')

joined = df.join(
    vertex_features, df['airport_id']==vertex_features['id'],
    'left'
)
```

continue with ML pipeline using the enriched DataFrame